

CAPABL Agentic AI Hackathon

Problem Statements Reference

Use: Give this document to an AI agent so it knows the available CAPABL problem statements and their exact requirements. This PDF is based only on the uploaded source conversation; missing information has not been invented.

Important: The uploaded source mentions Track 1 (Healthcare), but its full problem statement is not present in the file. Therefore this reference includes the detailed statements that are actually available: Tracks 2–6 and the Wildcard. Do not assume or invent the missing Track 1 details.

Track 3 — Education: Transcript & Study Plan Agent

EducationTranscript & Study Plan Agent

Scenario

Build an agent that a student hands their own transcript, syllabus, or set of course notes to, and the agent immediately identifies weak areas and builds a short prioritized study plan — before the student asks anything. It then answers questions grounded in the material, and separately searches for job or internship listings matched to what the transcript actually shows the student has studied, not a generic keyword the student types in.

Data to source

A real or sample transcript, syllabus, or set of course notes. A jobs/internship API (Adzuna or similar).

The two required tools, defined precisely: Study plan generator tool — runs automatically the moment the transcript or notes are ingested. Deterministic logic that flags topics with low grades or marks, or topics listed in a syllabus but absent from the notes, and outputs a short prioritized study plan. This automatic-on-ingestion behavior is the core deliverable of this track. Job/internship search tool — searches live listings using keywords extracted from the transcript or coursework itself, not a keyword the student has to supply.

Build instructions: LLM setup — working ChatOllama call first. Document loading — load the student's own transcript, syllabus, or notes with PyPDFLoader or a text loader. Text splitting — chunk by topic/section where possible so a course, grade, and topic list stay together rather than being split across chunks. Embeddings + vector store — embed and persist to Chroma, scoped to this student's own uploaded material. Retriever — confirm it retrieves the right section for both narrow questions ("what grade did I get in databases") and broader ones ("what have I not covered yet") before moving on. Custom tools — build the study plan generator and the job/internship search tool exactly as defined above. Tool calling — decorate with @tool; docstrings should make clear the search tool draws its keywords from the transcript, not from free-typed input. Agent — build the agent so it runs the study plan generator automatically on ingestion, presents the weak-area flags and prioritized plan before any question is asked, and only then answers follow-up questions or runs an opportunity search using topics pulled from the transcript. Deployment — Gradio or Streamlit app built around a transcript upload, with the study plan shown immediately after upload.

What the finished app must do, end to end

The student uploads their own transcript or notes. Before any question is typed, the agent surfaces weak-area flags and a short prioritized study plan. Only after that does the student ask follow-up questions or request a matched opportunity search.

Bonus (optional)

Serve both tools from a FastMCP server on its own port and have the agent fetch them at runtime via MultiServerMCPClient.

What judges look for

Whether the agent proactively surfaces a study plan on upload instead of waiting to be asked, whether the weak-area extraction logic is sound, and whether opportunity search results are genuinely matched to the transcript's actual content rather than generic keywords.

Turn 6 — ■ Assistant

Read against your problem statement, this is what the track actually demands.

Core deliverable: the study plan appears on upload, before any question is asked. Judges check three things: whether the plan is proactive, whether the weak-area logic is sound, and whether job results come from transcript content and not typed keywords.

What does what

LayerHandlesDeterministic PythonParsing courses and grades, weak-area flags, priority scoring, syllabus-vs-notes gap, keyword derivation, Adzuna callRAGAnswering "what grade did I get in databases" and "what haven't I covered", with the chunk shown as the sourceLLM (ChatOllama)Choosing tools and phrasing answers. It must not compute the plan or pick the search keywords.AgentA ReAct loop over 3 tools: search_student_material (the retriever), get_study_plan, search_opportunities
Decisions I'd make

Auto-run on upload: the upload handler calls the plan tool directly (generate_study_plan.invoke(...)), and Gradio renders the result. Don't wait for the LLM to decide to call it. The result is also inserted into the agent's message history, so follow-up questions can refer to it.

Input format: real transcripts vary too much to parse in one day, and PyPDFLoader does no OCR, so scanned PDFs won't work. Write a sample transcript, syllabus, and notes in one consistent layout. The spec allows sample data. Support a text-based PDF and .txt only, and cut everything else.

Plan logic:

Normalize grades to a percentage.

Flag courses below a threshold (my proposal: under 50 is High, 50–64 is Medium).

Score by (threshold gap × credits).

Add syllabus topics that are missing from the notes, using fuzzy string matching (rapidfuzz) so the result is explainable.

Show the top 5.

The thresholds are my assumption, not from your rules. Put them in one config block and mention them in the demo.

Search keywords: the tool takes no keyword argument. It reads the stored transcript and uses the student's strongest completed courses and topics, since those show what they've studied. It returns each listing with the keywords that matched it. If results are empty, retry once with fewer keywords, then say so plainly.

Chunking: one chunk per course record (course, grade, topics together) via a regex on course blocks. Use a header-based splitter for notes. Add a Chroma collection per upload.

Priority

Must:

ChatOllama with a tool-calling model

Loader

Course-block chunking

Embeddings and Chroma

Retriever tested on a narrow and a broad question

Plan tool

Job tool

@tool docstrings

ReAct agent

Gradio: upload → plan table → chat locked until the plan shows

Handling for empty retrieval, API failure, and unparseable files

Should:

Deterministic get_course_grade tool. Grade lookups shouldn't depend on retrieval alone.

Chat and job buttons appear only after the plan.

Cache the last real Adzuna response for demo failures, labeled as cached.

Nice: FastMCP bonus, only after everything else works end to end.

Cut: auth, a database beyond Chroma, OCR, reranking, multi-student support, charts, and any fine-tuning.

Costs

Ollama, Chroma, Gradio: free and open-source. Ollama needs enough local RAM for a 7–8B model. I haven't checked your machine.

Adzuna API: the terms give default limits of 25 hits per minute, 250 per day, 1000 per week and 2500 per month, and a third-party listing describes the API as free with rate limits. Auth is an app_id and app_key pair, and it covers 16 markets, including in. I didn't verify whether signup needs a credit card, or how many internship listings India has. Check both before building around it. terms of service +3

Hosting: run it locally. A free hosted Space won't have your Ollama model. Nothing in the text you pasted requires a public URL.

Risks

Small local models are unreliable at tool calling. Test qwen2.5:7b and llama3.1:8b with a bare tool call before building anything else. If neither works, the whole agent is at risk.

Adzuna keyword parameter: I believe it supports an OR-keywords parameter (what_or), but I haven't verified it. Check the docs first.

Agent framework: LangGraph's prebuilt ReAct agent and LangChain's newer agent constructor have both changed recently. Use whichever one your installed version documents.

Demo in 3 minutes

Upload the sample files. The plan appears without a question.

Ask "what grade did I get in databases" (retrieval).

Click "find opportunities" (tool call, with keywords shown).
Explain one design decision: the plan is deterministic code, not LLM output.

Do this first

Start Ollama, run one ChatOllama call, and run a bare tool-calling test on your candidate model.
Register for Adzuna and make one manual API call.
Write the sample transcript, syllabus, and notes. Every later step depends on their format.

Ask me for step 3 first if you want, and I'll draft the sample files and the parser together.

Turn 7 — ■ You

show is there a presentation of ppt

Turn 8 — ■ Assistant

No. The Track 3 text you pasted mentions no PPT or slide deck, and no file is attached to this chat. The only presentation-like requirement I've seen is the 3-minute demo, which comes from your own instructions, not from the track text.
The general CAPABL rules may say something about slides or submission format, and I haven't seen them. Check the rules page or your registration email for "submission", "deliverables", or "presentation". If you paste that section here, I'll tell you exactly what's required.
If a PPT does turn out to be required, I can outline the slides from the demo flow. Otherwise, start on the three "do this first" steps.

Turn 9 — ■ You

so before doing anything tell me what ur doing

Turn 10 — ■ Assistant

So far I've only answered from the text you pasted: no files created, no code run. Two other actions happened. I ran a web search to check Adzuna's limits, and I saved a short note about your hackathon track to the project's memory.
If you say go, this is what I'll do next, in order:

Write three sample files. I'll make a transcript, a syllabus, and course notes in one fixed format. They'll have deliberate gaps, such as a low grade and a syllabus topic missing from the notes, so the weak-area logic has something to find.
Write the parser and the study plan tool in Python. This is the deterministic part: grade normalization, weak-area flags, priority scoring, and the syllabus-vs-notes gap check.
Test that part in my sandbox and show you the real output.

I can't test the rest here. The sandbox has no network and probably no Ollama, so ChatOllama, Chroma with Ollama embeddings, and the Adzuna call have to be tested on your machine. I'll write that code only when we get to it, and I'll mark it as untested until you run it.
Say "go" to start step 1, or tell me what to change first.

Turn 11 — ■ You

ok go

Turn 12 — ■ Assistant

Steps 1–3 are done. I ran the deterministic core in my sandbox, and all 8 tests pass. Nothing else is tested yet.
What exists

Sample data: a 6-course transcript, a syllabus, and notes. The gaps are deliberate: two low grades, one borderline grade, and syllabus topics missing from the notes.

study_core.py: plain Python with no dependencies. It parses the three files, normalizes marks and letter grades to a percentage, flags weak courses, finds syllabus topics missing from the notes, and returns a top-5 prioritized plan. Thresholds and weights are in one CONFIG block.

Tests: they cover parsing, flags, gaps, plan ordering, missing syllabus or notes, an all-good student, malformed files, and a typo in a course name.

One bug I found and fixed: the first plan listed a course twice, once as weak and once as a coverage gap. It now shows one item per course. The tests caught the ordering, but I only spotted the duplication by reading the printed plan.

Limits

The letter-grade map and the 50/65 thresholds are my assumptions. Change them in CONFIG if your college's scale differs.

The parser only reads this fixed format. Real transcripts need conversion to it first.

A topic counts as covered if it appears in the notes text or a similar heading. That can miss synonyms, such as "indexes" vs "Indexing", and it can also mark a topic as covered when the notes only mention it in passing.

Not built or tested: ChatOllama, Chroma and embeddings, the retriever, the @tool wrappers, the agent, the Gradio app, and Adzuna.

Next step: the RAG layer. That means loading the files, chunking one course per chunk, and building Chroma with nomic-embed-text. I can write it, but you'll have to run it, since it needs Ollama. Before that, please run ollama pull for a model and tell me which one you get working, ideally with one successful tool call. Then I'll write the RAG code for that exact setup.
Study corePY Test study corePY TranscriptTXT SyllabusTXT NotesDocument · MD

Turn 13 — ■ You

Track 2 — Finance: Portfolio & Statement Simplifier Agent

FinancePortfolio & Statement Simplifier Agent

Scenario

Build an agent that a user hands their own brokerage statement or portfolio holdings sheet to, and the agent immediately analyzes it — summarizing performance and risk in plain language and flagging concentration or underperformance — before answering any question. It then stays available to answer follow-ups, pulling in live market data to keep answers current.

Data to source

A sample brokerage/fund statement or a portfolio holdings sheet you create (ticker, quantity, purchase price, current price). A free stock market data API (Alpha Vantage, Yahoo Finance, or similar) for live prices.

The two required tools, defined precisely
Portfolio analysis tool — runs automatically the moment the statement is ingested. Deterministic Python logic that computes gain/loss per holding, concentration percentage per position, and flags anything over a threshold you define (for example, any single stock over 25% of the portfolio). This automatic-on-ingestion analysis is the core deliverable of this track.
Live market data lookup tool — takes a ticker and returns current price or recent news, used to keep the agent's answers grounded in today's numbers rather than the statement's numbers alone.

Build instructions
LLM setup — get a working ChatOllama call first.
Document loading — load the user's own statement or holdings sheet (PDF or spreadsheet) with the appropriate LangChain loader.
Text splitting — statements have tables and dense numeric sections; test chunk_size/chunk_overlap carefully so figures stay attached to their labels.
Embeddings + vector store — embed and persist to Chroma, scoped to this one uploaded statement.
Retriever — confirm as _retriever(search_kwargs={"k": 3}) returns the right holding or section for a specific query ("how is my tech position doing") before moving on.
Custom tools — build the portfolio analysis tool and the live market data tool exactly as defined above. The analysis tool must be real computation on the parsed holdings, not an LLM estimate.
Tool calling — decorate with @tool; docstrings must state units clearly (INR vs USD, percentage vs absolute) since financial answers are wrong if units are ambiguous.
Agent — build the agent so it runs the portfolio analysis tool automatically on ingestion, presents a plain-language performance and risk summary with flags before any question is asked, and only then answers follow-ups — combining the uploaded statement with live data and always stating which source a number came from.
Deployment — Gradio or Streamlit app built around a statement upload, with the risk summary shown immediately after upload. Add a visible disclaimer: this is not personalized financial advice.

What the finished app must do, end to end

The user uploads their own portfolio statement. Before any question is typed, the agent computes and presents a plain-language performance and risk summary with concentration flags. Only after that does the user ask follow-up questions, answered using the statement plus live market data.

Bonus (optional)

Serve both tools from a FastMCP server on its own port and have the agent fetch them at runtime via MultiServerMCPClient.

What judges look for

Whether the agent proactively surfaces risk and concentration flags on upload instead of waiting to be asked, correctness of the underlying math in the analysis tool, and correct distinction between live and statement-sourced numbers in every answer.

Turn 14 — ■ Assistant

You didn't say what you want done with Track 2, so I'm assuming you're choosing between the two tracks. If that's wrong, tell me. My recommendation is to stay on Track 3. The two tracks share the same pipeline (loader, splitter, Chroma, retriever, agent, Gradio). What differs is the parser, the two tools, and the external API.

Track 3 (Education)Track 2 (Finance)Work already doneDeterministic core built and testedNothing yet, so the parser and tools start from zeroCore mathCustom scoring rules, needs justificationSimple: gain/loss and concentration %. Easy to get right, and judges check the math.InputFixed-format textA CSV or spreadsheet holdings sheet is easy to load. A real brokerage PDF is not.External APIAdzuna: 250 requests/day, 2,500/month (from my earlier check)Alpha Vantage free key: 25 requests per day and 5 per minuteExtra riskAdzuna may return few internship listingsUnits: INR vs USD, and whether Indian tickers are covered (unverified). The docstrings must state units.

The API limit is the deciding factor for Track 2. A 10-holding portfolio uses 10 of your 25 daily requests per refresh, so a few test runs plus the demo could exhaust the key. Yahoo Finance access through yfinance avoids that limit, but it's unofficial. I haven't verified whether it currently works or how it's rate-limited, so treat it as unconfirmed. Track 2's analysis tool doesn't need live data, since the sheet already has current prices. The problem is the follow-up questions, which do.

If you hadn't started, the two tracks would be close. Track 2 has easier math and harder live data. What tips it to Track 3 is that you

have less left to build.

Two things I can't tell from here: whether the rules let you switch tracks, and whether your team already has an assigned one. Check both before deciding.

Which track are you actually assigned or committed to?

Turn 15 — ■ You

no so the thing is there are 7 problem statements and in that first we choose the education and now we are thinking of selecting finance i am giving u all the problem statements tell me which one is the best Track 2: FinancePortfolio & Statement Simplifier Agent

Scenario

Build an agent that a user hands their own brokerage statement or portfolio holdings sheet to, and the agent immediately analyzes it — summarizing performance and risk in plain language and flagging concentration or underperformance — before answering any question. It then stays available to answer follow-ups, pulling in live market data to keep answers current.

Data to source

A sample brokerage/fund statement or a portfolio holdings sheet you create (ticker, quantity, purchase price, current price). A free stock market data API (Alpha Vantage, Yahoo Finance, or similar) for live prices.

The two required tools, defined precisely
Portfolio analysis tool — runs automatically the moment the statement is ingested. Deterministic Python logic that computes gain/loss per holding, concentration percentage per position, and flags anything over a threshold you define (for example, any single stock over 25% of the portfolio). This automatic-on-ingestion analysis is the core deliverable of this track.
Live market data lookup tool — takes a ticker and returns current price or recent news, used to keep the agent's answers grounded in today's numbers rather than the statement's numbers alone.

Build instructions
LLM setup — get a working ChatOllama call first.
Document loading — load the user's own statement or holdings sheet (PDF or spreadsheet) with the appropriate LangChain loader.
Text splitting — statements have tables and dense numeric sections; test chunk_size/chunk_overlap carefully so figures stay attached to their labels.
Embeddings + vector store — embed and persist to Chroma, scoped to this one uploaded statement.
Retriever — confirm as_retriever(search_kwargs={"k": 3}) returns the right holding or section for a specific query ("how is my tech position doing") before moving on.
Custom tools — build the portfolio analysis tool and the live market data tool exactly as defined above. The analysis tool must be real computation on the parsed holdings, not an LLM estimate.
Tool calling — decorate with @tool; docstrings must state units clearly (INR vs USD, percentage vs absolute) since financial answers are wrong if units are ambiguous.
Agent — build the agent so it runs the portfolio analysis tool automatically on ingestion, presents a plain-language performance and risk summary with flags before any question is asked, and only then answers follow-ups — combining the uploaded statement with live data and always stating which source a number came from.
Deployment — Gradio or Streamlit app built around a statement upload, with the risk summary shown immediately after upload. Add a visible disclaimer: this is not personalized financial advice.

What the finished app must do, end to end

The user uploads their own portfolio statement. Before any question is typed, the agent computes and presents a plain-language performance and risk summary with concentration flags. Only after that does the user ask follow-up questions, answered using the statement plus live market data.

Bonus (optional)

Serve both tools from a FastMCP server on its own port and have the agent fetch them at runtime via MultiServerMCPClient.

What judges look for

Whether the agent proactively surfaces risk and concentration flags on upload instead of waiting to be asked, correctness of the underlying math in the analysis tool, and correct distinction between live and statement-sourced numbers in every answer. ← Prev4 / 10Next →

Track 4 — Legal: Contract Simplifier & Compliance Agent

LegalContract Simplifier & Compliance Agent

Scenario

Build an agent that a user hands their own contract or policy document to, and the agent immediately rewrites dense clauses into plain English and runs a compliance checklist — flagging missing or non-standard clauses before any question is asked. It then answers follow-up questions about specific clauses, always citing the exact text it's answering from.

Data to source

Public contract templates, NDAs, or terms-of-service documents (many are freely available for practice; do not use real confidential contracts). A checklist of clauses/deadlines you define yourselves as the "compliance rules."

The two required tools, defined precisely: Clause simplification & checklist tool — runs automatically the moment the contract is ingested. Rewrites retrieved clauses into plain English and checks them against your defined rules using deterministic Python logic (regex/keyword checks for required clause types), returning pass/fail/missing for each. This automatic-on-ingestion behavior is the core deliverable of this track. Retrieve contract text tool — wraps your retriever so the agent can pull the exact clause text it's citing when answering a follow-up question.

Build instructions: LLM setup — working ChatOllama call first. Document loading — load the user's own contract or policy PDF with PyPDFLoader. Text splitting — legal documents are clause-structured; split on clause/section boundaries (numbered clauses) rather than a fixed character count, so a clause is never split across two chunks. Embeddings + vector store — embed and persist to Chroma, scoped to this one uploaded contract. Retriever — test that a query like "what is the termination notice period" retrieves the exact right clause, not an adjacent one. Custom tools — build the clause simplification & checklist tool and the retrieve-contract-text tool exactly as defined above. Tool calling — decorate with @tool; the checklist tool's docstring must clearly state what it checks for and what it returns (pass/fail/missing). Agent — build the agent so it runs the simplification & checklist tool automatically on ingestion, presents a plain-language summary with what's present and what's missing before any question is asked, and only then answers follow-up questions, citing the retrieved clause text it's answering from. Deployment — Gradio or Streamlit app built around a contract upload, with the plain-language summary and checklist results shown immediately after upload. Include a visible disclaimer: not a substitute for a licensed lawyer's review.

What the finished app must do, end to end

The user uploads their own contract. Before any question is typed, the agent presents a plain-language summary of key clauses and a compliance checklist showing what's present and what's missing. Only after that does the user ask follow-up questions, each answered with a citation to the retrieved clause.

Bonus (optional)

Serve both tools from a FastMCP server on its own port and have the agent fetch them at runtime via MultiServerMCPClient.

What judges look for

Whether the agent proactively summarizes and checks the contract on upload instead of waiting to be asked, whether the compliance checks are genuinely deterministic, and whether every follow-up answer cites the specific clause it's based on. ← Prev6 / 10Next →

Track 5 — E-commerce/Retail: Order & Warranty Agent

E-commerce/RetailOrder & Warranty Agent

Scenario

Build an agent that a shopper hands their own receipt or order confirmation to, and the agent immediately summarizes what was purchased and calculates the return or warranty window — flagging anything expiring soon before any question is asked. It then handles follow-up questions and can look up live order status.

Data to source

Sample receipts or order confirmations you create, a return/shipping policy document, and a mock order database (a simple CSV or SQLite table you seed yourselves) standing in for a real order system.

The two required tools, defined preciselyReturn/warranty window calculator tool — runs automatically the moment the receipt is ingested. Deterministic Python logic that computes days remaining in the return or warranty window from the purchase date and the policy document, and flags anything with fewer than 7 days left. This automatic-on-ingestion behavior is the core deliverable of this track.Order status lookup tool — queries your mock order database by order ID and must handle a not-found ID gracefully rather than fabricating a status.

Build instructionsLLM setup — working ChatOllama call first.Document loading — load the shopper's own receipt or order confirmation, plus the store's return/shipping policy document.Text splitting — one item per chunk works better than a fixed character split for receipts; chunk the policy document separately.Embeddings + vector store — embed and persist to Chroma, scoped to this shopper's own uploaded receipt.Retriever — confirm a query like "when can I return the jacket" retrieves the right item and the right policy clause.Custom tools — build the return/warranty window calculator and the order status lookup tool exactly as defined above.Tool calling — decorate with @tool; the calculator's docstring must state how it computes the window (purchase date + policy days).Agent — build the agent so it runs the window calculator automatically on ingestion, presents a plain-language purchase summary with any expiring-soon flags before any question is asked, and only then answers follow-ups or runs an order-status lookup — never recommending a product or policy term not actually present in the data.Deployment — Gradio or Streamlit app built around a receipt upload, with the summary and flags shown immediately after upload.

What the finished app must do, end to end

The shopper uploads their own receipt. Before any question is typed, the agent presents a plain-language purchase summary with return/warranty windows and flags anything expiring soon. Only after that does the shopper ask follow-up questions or check an order's live status.

Bonus (optional)

Serve both tools from a FastMCP server on its own port and have the agent fetch them at runtime via MultiServerMCPClient.

What judges look for

Whether the agent proactively flags expiring windows on upload instead of waiting to be asked, correctness of the date math, and whether the agent ever hallucinates a product, policy term, or order that isn't actually in the data.← Prev7 / 10Next →

Track 6 — HR/Recruitment: Resume & Offer Letter Agent

HR/RecruitmentResume & Offer Letter Agent

Scenario

Build an agent that a candidate or recruiter hands an actual resume and job description (or offer letter) to, and the agent immediately produces a structured match-and-gap summary the moment both documents are in — before any question is asked. It then answers follow-up questions and can check interview slot availability.

Data to source

A company handbook/benefits document (real or mocked), a real job description, a real resume (yours or a teammate's, with consent), and a mock interview-slot calendar (a simple structure you seed yourselves).

The two required tools, defined preciselyResume-to-JD match tool — runs automatically the moment both the resume and job description have been ingested. Retrieves chunks from both and returns a structured comparison — matched skills, missing skills — as deterministic output, not a vague opinion. This automatic-on-ingestion behavior is the core deliverable of this track.Interview slot lookup/booking tool — queries your mock calendar for availability.

Build instructionsLLM setup — working ChatOllama call first.Document loading — load the handbook, the actual job description, and the actual resume with PyPDFLoader.Text splitting — chunk the handbook/JD with RecursiveCharacterTextSplitter; keep resume chunking separate from policy chunking, tagging each chunk's document type so retrieval doesn't mix candidate data with company policy.Embeddings + vector store — embed and persist to Chroma, with metadata distinguishing "policy," "resume," and "job description" chunks.Retriever — confirm you can retrieve policy-only, resume-only, or JD-only chunks depending on the question.Custom tools — build the resume-to-JD match tool and the interview slot tool exactly as defined above.Tool calling — decorate with @tool; the match tool's docstring should specify it returns a structured comparison, not a vague opinion.Agent — build the agent so it runs the match tool automatically once both the resume and JD are ingested, presents the matched/missing skills summary before any question is asked, and only then answers policy questions, screening follow-ups, or scheduling requests — keeping the three flows clearly separated.Deployment — Gradio or Streamlit app built around a resume + JD upload, with the match summary shown immediately after both are in.

What the finished app must do, end to end

The candidate or recruiter uploads a resume and a job description. Before any question is typed, the agent presents a structured matched/missing-skills summary. Only after that do policy questions, screening follow-ups, or scheduling requests get answered, and each flow stays cleanly separated from the others.

Bonus (optional)

Serve both tools from a FastMCP server on its own port and have the agent fetch them at runtime via MultiServerMCPClient.

What judges look for

Whether the agent proactively produces the match summary as soon as both documents are in, whether policy, screening, and scheduling stay cleanly separated with no data bleed, and match-tool output quality (structured, not vague).← Prev8 / 10Next →

Wildcard — Meme & Internet Culture Historian

Wildcard TrackMeme & Internet Culture Historian

Scenario

Build an agent that acts as a walking encyclopedia of internet culture — but not a passive one. On demand it proactively compiles a "what's trending right now and why" digest by cross-referencing a live trending-topics feed against its own retrieved knowledge of meme history, then stays available to answer specific lookup questions grounded in that same knowledge base.

Data to source

A meme/trend reference set the team compiles themselves (Know Your Meme entries, Wikipedia's internet-culture pages, or a curated doc with origin story, first appearance, meaning, and notable variations for 30 to 50 memes/trends). A live trending-topics source (X/Twitter trends API, Google Trends, Reddit's trending endpoint, or a news-headlines API as a proxy).

Build instructionsLLM setup — working ChatOllama call first.Document loading — load the meme/trend reference documents (PDF, markdown, or scraped text you compile).Text splitting — one meme/trend per chunk works better than a fixed character split; each entry is short and self-contained, so keep origin, meaning, and timeline together in the same chunk rather than letting the splitter cut through one.Embeddings + vector store — embed and persist to Chroma.Retriever — confirm a fuzzy or partial-name query ("that skibidi thing" or "the distracted boyfriend guy") still retrieves the right entry; meme names are inconsistent and slangy, so this is a genuine retrieval stress test, not a trivial one.Custom tools — build at least two: (a) a "live trending topics" tool that fetches what's currently trending from a real feed, (b) a "retrieve meme/trend history" tool wrapping your retriever.Tool calling — decorate with @tool; docstrings should make clear one tool is for historical/reference lookup and the other is for right-now data, so the agent doesn't confuse "what does this mean" with "what's trending today."Agent — build the agent so that on a "compile today's digest" request it proactively fetches several live trending topics and cross-references each against retrieved historical knowledge, producing a short digest with a for-each connection or an honest "this is genuinely new" — separately from that, it should still answer one-off lookup questions grounded in the same knowledge base.Deployment — Gradio or Streamlit, styled like a fun trivia/culture digest, with a clear action to generate today's digest rather than only a chat box.

Bonus (optional)

Serve both tools from a FastMCP server on its own port and have the agent fetch them at runtime via MultiServerMCPClient.

What judges look for

Whether the digest generation is genuinely proactive (compiles several trends in one action rather than one Q&A pair at a time), retrieval precision on oddly specific, inconsistently-named entries, and whether it plausibly connects a live trend back to its historical roots instead of just describing the two independently.← Prev9 / 10Next →

Track 1 — Healthcare

The uploaded source references Track 1 as Healthcare, but does not contain the full Track 1 problem statement. No scenario, data requirements, tools, build instructions, deployment details, or judging criteria are reproduced here because they are not available in the source.